

Производные типы

Бывают

1. Указатель
2. Ссылка

Указатель

Указатель - механизм, позволяющий хранить и обрабатывать адреса ячеек памяти

& - взятие адреса

8 байт на указатель в 64-разрядных платформах и 4 в 32-разрядных

Операции

1. Присваивание адреса
2. Разыменование (*****)
3. Операции сравнения
4. Адресная арифметика (**+** , **-** , **++** , **--**)

Адрес изменяется на размер типа данных, с которым производится операция

Динамические данные в куче (heap)

```
double* dx; // null pointer (нулевой указатель)
dx = new double;
delete dx;
```

Адреса для динамических переменных хранятся в стеке, данные в куче

Ссылка

Ссылка - механизм, позволяющий привязать к значению имя; псевдоним некоторой переменной

```
int a = 5;  
int& p = a;  
p == a; // true
```

Ссылка на константу

```
const double g = 9.8;  
const double& pg = g;  
  
double& grav = g; // Error  
pg = 1.6 // Error
```

Константная ссылка

```
int a = 5;  
const int& b = a;  
b = 10; // Error  
a = 10 // OK
```

Составные типы

Составной тип - тип, который объединяет несколько типов данных в себе

Виды

1. Массив (`array`)
2. Структура (`struct`)
3. Объединение (`union`)
4. Перечисление (`enum`)
5. Класс (`class`)

Массив

- Все данные лежат в одном месте, **одним блоком**
- Все элементы **проиндексированы**
- Все элементы **одного типа**

Чтение и запись элемента по индексу выполняется за $O(1)$

Массивы по размеру

1. одномерные
2. многомерные

Массивы также бывают

1. автоматические
2. динамические

Не рекомендуется писать так

```
int k;  
std::cin >> k;  
double t[k];
```

Определение размера массива

```
size_t length = std::size(array);
```

`size_t` - специальный беззнаковый тип данных для индексации и хранения размеров (длина массива, `i` в `for`)

При обращении по индексу можно выйти за границы массива (undefined behavior)

For-each (range-for)

```
// Можно самостоятельно указать тип вместо auto
for (auto a: arr) {...}
```

Переменная в `for-each` - это локальная копия элемента массива, то есть (обычно) массив в `for-each` только на чтение

Можно записывать в `for-each` через ссылки:

```
for (int& a: arr) {a = 2 * a}
```

Заполнение массива случайными числами

Старый способ

```
// Для чисел в диапазоне [a ; b]
rand() % (b - a + 1) + a
```

```
// Для действительных чисел в диапазоне [a ; b]
rand() % (b - a + 1) / RAND_MAX + a
```

Современный способ

```
#include <random>

random_device rd;
mt19937 gen(rd()); // mt19937 - специальный тип данных

// Для целых чисел из [a ; b]
uniform_int_distribution distInt(a, b)
a = distInt(gen);

// Для вещественных чисел из [a ; b]
uniform_real_distribution distReal(a, b)
a = distReal(gen);
```

Указатели и массивы

Имя массива - указатель на первый элемент массива

```
double arr {1, 2, 3};
arr[0] == *arr // True
```

Обход массива через указатель

```
for (double* ptr = arr; ptr <= arr + length; ptr++)
{...}
```

Разыменование двумерного массива

```
*(* (arr + i) + j)
```

Динамический массив

Одномерный

```
int n;
std::cin >> n;
double* arr = new double[n];
delete[] arr; // для каждого массива свой delete[]
```

Двумерный

```
// Выделяем память для указателей на строки
double** arr = new double*[n];

for (size_t i = 0; i < n; i++) {
    // Выделяем память под каждую строчку
    arr[i] = new double[m];
}
```

При таком подходе перестает работать принцип о непрерывности массива

```
for (size_t i = 0; i < n; i++) {
    // Освобождаем память под каждую строчку
    delete[] a[i];
}

// Освобождаем память под набор указателей на строки
delete[] a;
```

Многомерный массив

- a_i - одномерный массив
- a_{ij} - двумерный массив
- a_{ijk} - трехмерный массив

```
int a[m][n] = { {...}, {...}, ... , {...} }
```

(Дополнительно)

`setw(n)` - сколько символов будет занимать вывод (нужно подключить `<iomanip>`)